УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА У НОВОМ САДУ

Дамјан Манојловић

Симулација преживљавања популације јединки

ДИПЛОМСКИ РАД
- Основне академске студије -

Нови Сад, 2021

	УНИВЕРЗИТЕТ У НОВОМ САДУ ● **ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА** 21000 НОВИ САД, Трг Доситеја Обрадовића 6	Датум:
	ЗАДАТАК ЗА ИЗРАДУ ДИПЛОМСКОГ (BACHELOR) РАДА	Лист/Листова: 1/1

(Податке уноси предметни наставник - ментор)

Врста студија:	☒ Основне академске студије
Студијски програм:	**Софтверско инжењерство и информационе технологије**
Руководилац студијског програма:	**Проф. др Мирослав Зарић**

Студент:	**Дамјан Манојловић**	Број индекса:	**SW 20/2017**
Област:	**Софтверско инжењерство**		
Ментор:	**др Игор Дејановић**		

НА ОСНОВУ ПОДНЕТЕ ПРИЈАВЕ, ПРИЛОЖЕНЕ ДОКУМЕНТАЦИЈЕ И ОДРЕДБИ СТАТУТА ФАКУЛТЕТА ИЗДАЈЕ СЕ ЗАДАТАК ЗА ДИПЛОМСКИ (Bachelor) РАД, СА СЛЕДЕЋИМ ЕЛЕМЕНТИМА:

- проблем – тема рада;
- начин решавања проблема и начин практичне провере резултата рада, ако је таква провера неопходна;
- литература

НАСЛОВ ДИПЛОМСКОГ (BACHELOR) РАДА:

СИМУЛАЦИЈА ПРЕЖИВЉАВАЊА ПОПУЛАЦИЈЕ ЈЕДИНКИ

ТЕКСТ ЗАДАТКА:

Уз ослонац на програмске језике Go i Pharo имплементирати симулацију преживљавања популације јединки. Кориснички интерфејс реализовати на програмском језику Pharo. Подржати и веб варијанту интерфејса употребом vue.js библиотеке. Прикупљати и складиштити податке о догађајима током симулације у виду дневника (енг. log).

При изради користити препоручену праксу из области софтверског инжењерства.
Детаљно документовати решење.

Руководилац студијског програма:	Ментор рада:

Примерак за: ☐ - Студента; ☐ - Ментора

Садржај

1. Увод *5*

2. Технологије *6*

2.1 Go *6*

2.1.1 Gin 7

2.1.2 Goroutines 7

2.2 Pharo *8*

2.2.1 Roassal 9

2.2.2 Spec 10

2.2.3 Zinc 11

2.3 Vue.js *11*

2.3.1 Vuex 12

2.3.2 Vuetify 13

2.4 MongoDB *14*

2.5 CanvasJS *15*

3. Пројектовање и имплементирана техничка решења *16*

3.1 Модели података *16*

3.1.1 Храна, ресурси – извори хране 16

3.1.2 Јединка и популација 18

3.2 Однос и интеракција између модела *21*

3.3 Активности *24*

4. Приказ и начин имплементације пројектног задатка *27*

4.1 Go логичка јединица *27*

4.1.1 Структура логичке јединице 27

4.1.2 Занчајне функције 30

4.1.3 Паралелизам и његов ефекат 34

4.1.4 Интеракција са базом и логовима 35

4.2 Pharo кориснички интерфејс *37*

4.2.1 Кориснички интерфејс за унос параметара 38

4.2.2 Кориснички интерфејс за праћење симулације 39

4.3 Веб приказ статистике *40*

5. Закључак *42*

6. Литература *43*

1. УВОД

Тема овог рада јесте приказ пројекта који симулира преживљавања јединки, као и даља анализа добијених података приликом симулације. Преживљавање, у конкретном случају, представља процес приликом ког долази до егзистенцијалних потреба најпримитивнијих група јединки, као што су исхрана и репродукција. Укратко, преживљавање представља правилно коришћење природних ресурса у циљу очувања здравља, односно задовољавање основних животних потреба.

Пројекат се састоји из три кључна дела, два за приказ корисничког интерфејса и један за прикупљање, обраду и складиштења података. Кориснички интерфејси имплементирани су у две различите технологије, један је одрађен уз помоћ програмског језика Pharo, док је за други кориснички интерфејс коришћен Vue.js. Слој за манипулацију подацима, креирање дневника догађаја и само извршавање симулације реализовани у програмском језику Go. Дневник догађаја (eng. log) представља колекцију информација о догађајима који су се десили у току рада програма. Записују се и остају трајни на физичком меморијском медијуму. Свака од наведених целина ће касније бити детаљније објашњена.

Друго поглавље се бави технологијама које су коришћене приликом израде самог пројектног задатка. Значај другог поглавља огледа се у томе што је њихово познавање од круцијалног значаја за даље разумевање конкретног рада, те поглавље садржи информације и описе технологија које су употребљаване. Свака од технологија ће бити независно описана, као и њени радни оквири који су значајни за имплементацију пројектног задатка.

Треће поглавље се бави детаљним описом техничких решења, као и њиховом применом у оквиру конкретног пројектног задатка, са циљем да ближе прикаже пројектни задатак самом читаоцу. Заправо, бави се техничким аспектом самог пројекта, описује моделе података и њихову међусобну интеракцију. Приказује се дијаграм класа и дијаграм активности у оквиру једне интеракције у процесу преживљавања. Бави се такође описом активности унутар интеракције, као и њиховим значајем како за саму јединку, тако и за целокупну популацију.

Четврто поглавље се бави имплентацијом пројектног задатака и претходно објашњених техничких решења, као и визуелним приказом имплементације. Прецизније, бави се приказом како значајних делова кода, тако и корисничких интерфејса и објашњенима истих.

2. ТЕХНОЛОГИЈЕ

У овом поглављу ће бити описане технологије које су коришћене приликом реализације самог пројектног задатка. Значај овог поглавља огледа се у томе што је њихово познавање од круцијалне важности за даље разумевање конкретног рада, па поглавље садржи информације и описе технологија које су употребљаване. Свака од технологија ће бити независно описана, као и њени радни оквири (енг. frameworks) који су значајни за имплементацију пројектног задатка.

2.1 Go

Go програмски језик опште намене, развијен је у компанији *Google*. Ако узмемо у обзир да је настао 2007. године, а јавно представљен 2009. године, спада у младе програмске језике. Језик су осмислили Роберт Гриесемер, Роб Пике и Кен Тхомпсон, у покушају да укомбинују лак начин писања и читања интерпретираних језика и ефикасност и сигурност језика који се компајлирају.[1]

Go је настао и развија се под утицајем других језика, са циљем да се преузму добре идеје од других, али и да се избегну одлике које би могле да доведу до компликовања кода. Програмски језик је осмишљен тако да буде синтаксно сличан програмском језику C. Такође, може се рећи да припада породици C језика, али за разлику од програмско језик C поседује сигурност приликом приступа меморији (eng. memory safety). Go је статички типизиран, компајлиран програмски језик, има само 25 резервисаних речи, његови основни концепти су сведени на минимум, а његова синтакса је дизајнирана тако да буде читка и недвосмислена. Go своју најзначајнију примену тренутно има у домену изградње дистрибуираних система и микросервиса. Један од кључних фактора који то доказују је растућа заједница и веома активан развој. [2]

[1] Robert Griesemer, „The Evolution of Go”, Google Inc, 2015.

[2] „Frequently Asked Questions (FAQ)”, Golang, доступно: https://golang.org/doc/faq, [Последњи приступ 31. августа 2021]

2.1.1 Gin

Gin је HTTP веб радни оквир (енг. framework) високих перформанси написан у програмском језику Go. Gin осетно олакшава изградњу веб апликација и микросервиса у програмском језику Go. Такође, садржи скуп често коришћених функција, које смањују програмски код и поједностављују израду веб апликација, а неке од њих представљају подршку за: рендеровање, рутирарање, ткз. middleware подршка и многе друге. [3]

Приликом слања захтева, Gin прво парсира путању, ако се она поклапа, долази до хендловања руте и међукорака (eng. middleware), ако их има. На слици 1 је приказан типичан ток процесирања захтева у Gin.

```
Request -> Route Parser -> [Optional Middleware] -> Route Handler -> [Optional
Middleware] -> Response
```

Слика 1 - Ток процесуирања захтева у Gin

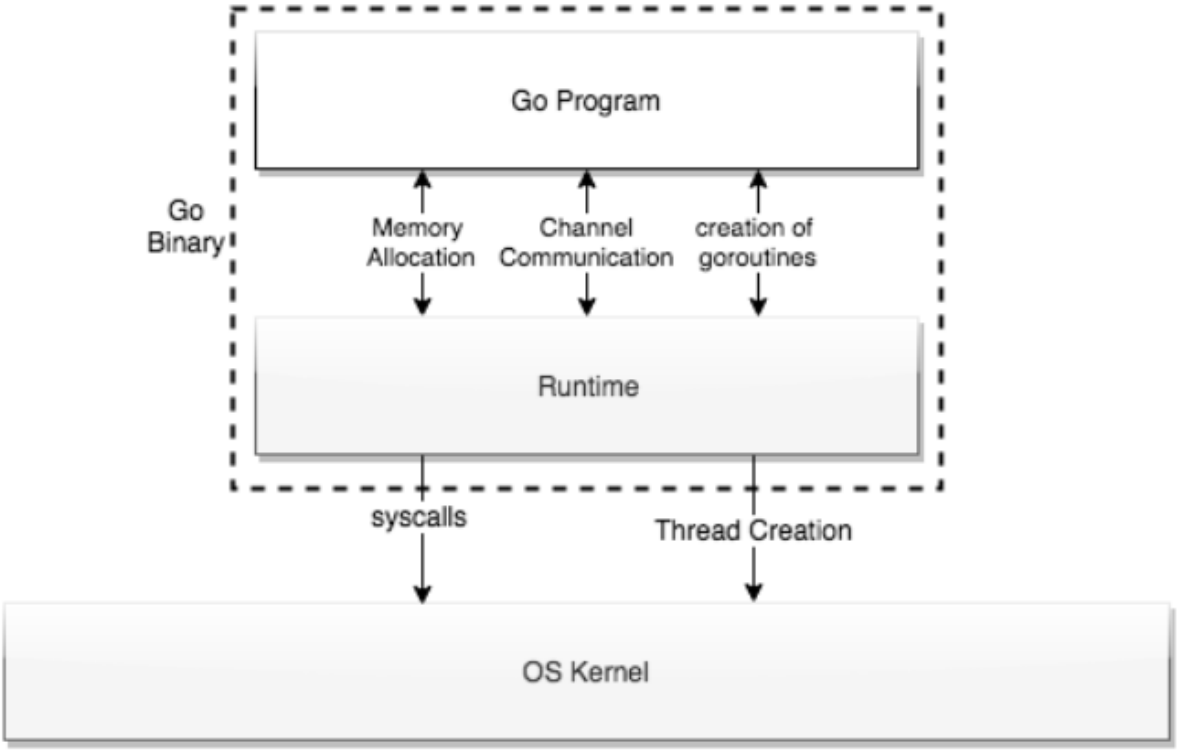
2.1.2 Goroutines

Конкурентност је постала један од битнијих фактора модерних софтвера. Go у себи поседује уграђену подршку за конкурентно програмирање, односно паралелизацију под називом Goroutines. Goroutine је функција или метода која се позива и извршава у паралели, независно од остатка програма. У програму може постојати више goroutine функција које се независно извршавају. Другим речима, свака активност која се извршава конкурентно у програмском језику Go, назива се goroutine.

Goroutine се креира коришћењем префикса *go* испред позива методе или функције. Међутим, још један битан елемент који омогућава конкурентност и који води рачуна о goroutines јесте *waitGroup. WaitGroup* се користи када не желимо из неког разлога (нпр. процеси зависе једни од других) да покренемо све процесе у паралели. Том приликом формирамо групу која чека на извршавање и тек након ње долази до извршавања

[3] „What is Gin in Golang?”, Eduactive, доступно: https://www.educative.io/edpresso/what-is-gin-in-golang, [Последњи приступ 31. августа 2021]

остатка програма. Односно, док се goroutines из конкретне групе не изврше, програм чека њихово извршавање (помоћу методе *Wait()*), да би наставио са даљим радом. На слици 2 је дијаграм који приказује однос између Go програма, ток извршавања и кернела оперативног система, који појашњава процес креирања goroutines, односно тредова.[4]



Слика 2 - однос између Golang програма, ток извршавања и кернела оперативног система

2.2 Pharo

Pharo је динамички језик отвореног кода, инспирисан *smalltalk* језиком. Једна од значајних особина програмског језика Pharo је та што садржи и интегрисано развојно окружење тј. IDE. Такође, комплетно је објективно орјентисан језик, који нуди директну манипулацију објектима у току извршавања (енг. runtime). Поседује виртуелну машину која је готово у потпуности написана у програмском језику Smalltalk. Она омогућава да буде преносив и подржава широк спектар оперативних система на којима може

[4] „Golang Concurrency”, Golang Programs, доступно: https://www.golangprograms.com/go-language/concurrency.html, [Последњи приступ 31. августа 2021]

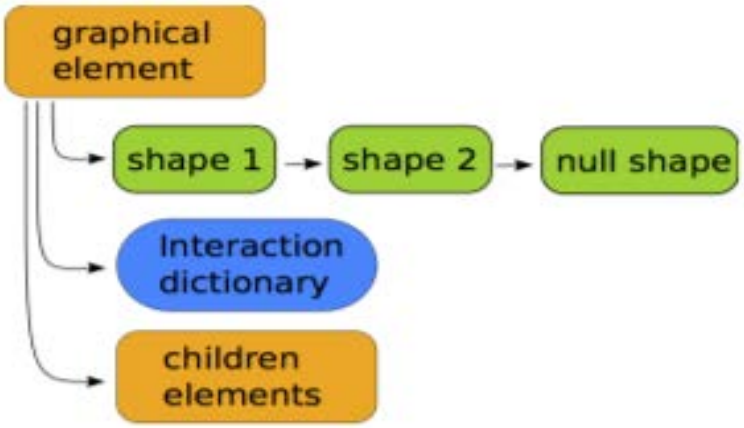
бити покренут, неки од њих су: OS X, windows, linux, android, iOS, и raspberryPi OS. [5]

Pharo поседује јако малу синтаксу што га чини интуитивнијим, односно олакшава савлађивање синтаксе истог. Једна од његових карактеристика је и та што су све методе су јавне и виртуелне, сви атрибути су заштићени и подржава само једнострука наслеђивања.[6]

2.2.1 Roassal

Roassal је агилни механизам направљен за визуелизацију и интеракцију са произвољним подацима, који је развијен и одржаван у програмском језику Pharo.

Коренска класа сваке визуелизације је ROView класа, она је контејнер за све графичке елементи које се приказују. Елементи се могу произвољно додавати, модификовати и уклањати из ROView објекта. Већина визуелизација садржи један *view* елемент у коме се налазе сви остали елементи. На слици 3 је приказана произвољан ROElement. [6]



Слика 3 - ROElement репрезентација

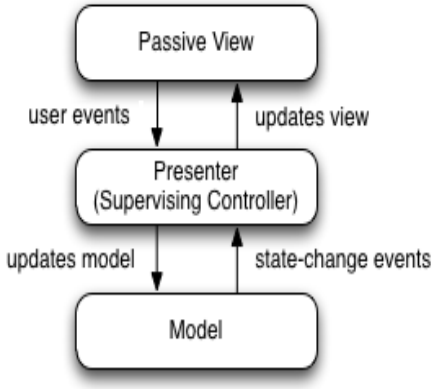
[5] Oscar Nierstrasz, Stéphane Ducasse, Damien Pollet, „Pharo by Example”, Lulu. com, 2010.

[6] Alexandre Bergel, Damien Cassou, Stéphane Ducasse, Jannik Laval, „Deep into Pharo”, Lulu. com, 2013.

2.2.2 Spec

Spec је оквир за развоје (енг. framework) који је израђен у програмском језику Pharo за описивање корисничких интерфејса. Он омогућава да се изгради велики број корисничких интерфејса, од малих прозора са неколико дугмади, па све до сложених алата дибагера. Чак је и више алата у самом програмском језику Pharo написано коришћењем радног оквира Spec, а неке од њих су: Watchpoint window, Komitter, Change Sorter, Critics Browser, Pharo 3 debugger, и многе друге.

Основни принцип који стоји иза радног оквира Spec је поновна употреба корисничких интерфејса. Наиме, кориснички интерфејси се граде поновном употребом и састављањем од већ постојећих корисничких интерфејса, спајајући и конфигуришући их по потреби. У основи, приликом писања конкретног spec кода, програмер не долази у директни контакт са виџетима корисничког интерфејса, уместо тога ткз. ComposableModel је програмиран тако да он садржи логику за креирање самог корисничког интерфејса. Када се кориснички интерфејс отвори, овај модел ће затим створити одговарајуће компоненте. За саме програмере ова разлика није заиста очигледна и осећа се као да се виџети програмирају директно. Такође омогућава раздвајање између различитих делова корисничког интерфејса по MVP (Model–view–presenter) обрасцу. [7] MVP образац је изведен из MVC (Model-view-controller) обрасца и један је од најчешћих образаца који се користе приликом изградње корисничког интерфејса. На слици 4 је приказан дијаграм који јасније показује структуру MVP обрасца.



Слика 4 - структура MVP обрасца

[7] Johan Fabry, Stéphane Ducasse, „The Spec UI framework”, Square Bracket Associates, 2017.

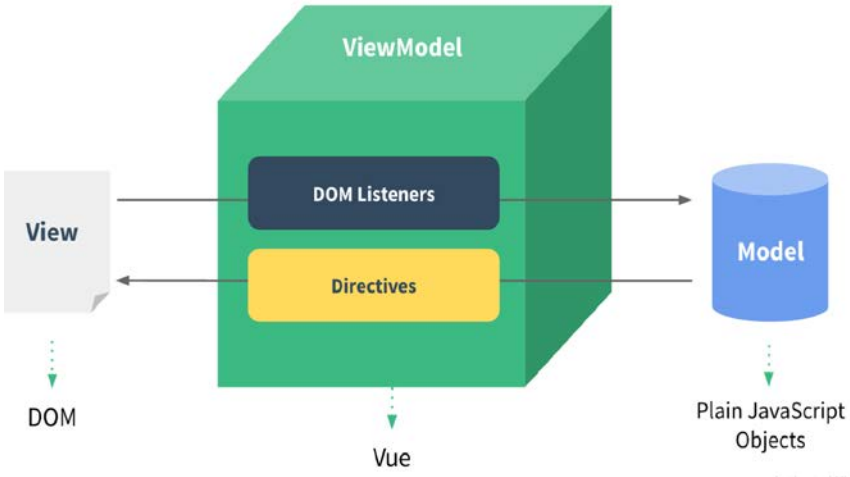
2.2.3 Zinc

Zinc је оквир за развоје (енг. framework) који је израђен у програмском језику Smalltalk, отвореног кода који се бави HTTP мрежним протоколима. Поседује способност да ради и манипулацију са већином HTTP концепата и сродних стандарда. Такође, zinc нуди како клијентске, тако и серверске функционалности. Један од његових примарних циљева је да пружи разумљивост. Свако ко има основно разумевање HTTP принципа је у стању да разуме шта се дешава и научи, гледајући имплементацију.

Кључне класе овог радног оквира су *ZnClient* и *ZnServer*. *ZnClient* је кључни објекат за програмско извршавање HTTP. Служи за конфигурисање и извршавање HTTP захтева и приступ одговору. *ZnClient* је објекат градитељ (енг. builder), док се инстацирањем *ZnServer* објекта креира HTTP сервер. [8]

2.3 Vue.js

Vue.js (у даљем тексту vue) је MVVM (Model–view–viewmodel) frontend javascript оквир (енг. framework) отвореног кода (енг. open source). На слици 5 је приказан дијаграм који јасније показује структуру MVVM обрасца. Његова намена и примена огледа се у креирању корисничких интерфејса и једностаничних веб апликација (енг. single-page applications). [9]



Слика 5 – структура Vue MVVM обрасца

[8] Sven van Caekenberghe, „Zinc: an industrial quality level HTTP/S framework", Square Bracket Associates, 2020.

[9] Macrae, Callum, „Vue. js: Up and Running: Building Accessible and Performant Web Apps", O'Reilly Media, Inc, 2018.

Vue поседује прилагодљиву архитектуру са могућношћу постепеног проширивања, како би се постепено усвајали напреднији елементи. Основна библиотека фокусирана је само на слоју приказа (енг. view layer), која има могућност интегрисања са другим библиотекама. Vue поседује неке од напреднијих функционалсти као што су рецимо рутирање и state менаџмент, а који су потребни за креирање сложенијих апликација.

Такође, vue омогућава проширење HTML-a са HTML атрибутима, који се називају директиве (енг. directives). Директиве су команде које можемо закачити на ДОМ елемент, користе се за low-level приступ HTML елементу, и његову контролу понашања. Директиве могу бити уграђене или кориснички дефинисане.

Још једну од кључних елемената програмског језика vue представљају компоненте. Компоненте су једна од најважнијих функција у Vue и представљају начин да се разбије комплексност апликације на мање делове. Компоненте нам помажу да енкапсулирамо код који се може користити више пута у апликацији. Сваки кориснички дефинисан HTML таг који представља инстанцу компоненте мора да има конструкторску функцију на основу које се генерише. [10]

2.3.1 Vuex

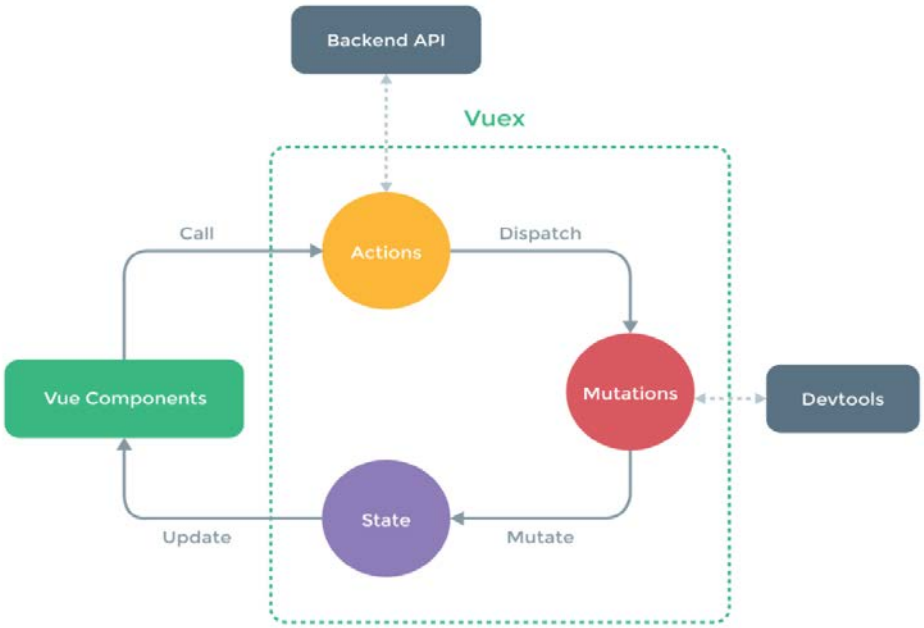
Vuex је state management образац у оквиру vue.js радног оквира. Vuex је стандардни додатак у vue апликацијама, који нуди централизовано складиште за приступ и манипулацију подацима у самој апликацији.

Унутар радног оквира Vuex апликација складишти све дељене податке, односно стања у апликацији. Стања унутра апликације се мењају помоћу мутација које се изводе као одговор на акцију која позива догађај саме мутације преко диспечера.

Компоненте могу имати локалне податке о којима само они морају знати. На пример, положај траке за померање у компоненти листе корисника вероватно не занима друге компоненте. Али било који податак који треба да се дели између компоненти, тј. подаци о апликацији, мора се чувати на једном месту, одвојено од компоненти које га користе. Компоненте могу

[10] Nelson, Brett, „Getting to Know Vue. js”, Apress, 2018.

слободно читати податке из продавнице, али не могу да мењају податке у store, барем не директно. Уместо тога, морају обавестити продавницу о својој намери да промене податке и складиште ће бити одговорно за те промене путем скупа дефинисаних функција које се зову мутације. На слици 6 је приказано место vuex компоненте унутар vue апликације. [11]



Слика 6 - Vuex унутар vue апликације

2.3.2 Vuetify

Vuetify је оквир (eng. framework) намењен за креирање корисничких интерфејса и изграђен је у Vue.js. Vuetify користи mobile first приступ дизајну, што значи да апликација једноставно функционише, било да се ради о телефону, таблету или стоном рачунару. Vuetify је складу са Material Design спецификацијом. То подразумева да ће у vuetify бити подразумеване доступне карактеристике из Vue и Material. [12]

---

[11] Andrea Koutifaris, „Vuex Quick Start Guide: Centralized State Management", O'Reilly Media, Inc, 2018.

[12] „Introduction - What is Vuetify?", VuetifyJs, доступно: https://vuetifyjs.com/en/introduction/why-vuetify/, [Последњи приступ 1. септембра 2021]

2.4 MongoDB

MongoDB је документ орјентисан NoSQL база података. NoSQL база података су базе података које не користе SQL за повезивање, нерелационе су, дистрибуиране, отвореног кода и хоризонтално скалабилне. MongoDB је написан у C++ програмском језику. MongoDB подразумевано чува податке у JSON (JavaScript Object Notation) документе са динамичким шемама.

MongoDB складишти колекције, а документа се складиште у колекцијама. Примарна разлика између докумената и колекција је у томе што су колекције неструктуиране. Структура инстанца колекције MongoDB није унапред дефинисана, за разлику од релационих база које складиште податке по строго дефинисаним правилима. Такорећи, MongoDB колекције примају све податке који се у њих унесу.

Документ је скуп кључ – вредност поља. Структура документа се може мењати у складу са подацима, што чини MongoDB лакшим и агилнијим за кодирање у односу на релационе базе података. Односно, интеракција између апликације и базе података ја бржа, али и непредвидљивија. [13]

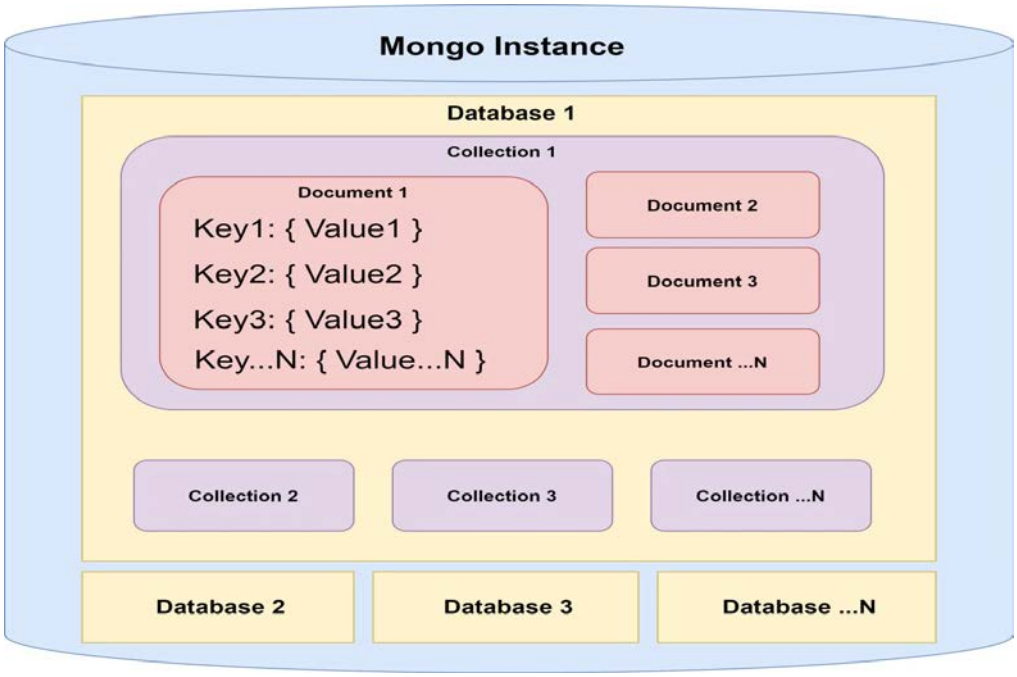
MongoDB је база података за општу употребу. Њено поседовање динамичке шеме података и објектно орјентисану структуру допринели су томе да је постала једна од водећих база података када су у питању пројекти везани за аналитику у реалном времену, као и за е-комерц, мобилне апликације, архивирање. Такође, своју улогу је нашла и у раду са великим подацима (енг. big data), кроз управљање садржајем, мобилну и друштвену инфраструктуру и многе друге. [14]

Слика 7 даје виртуелни приказ структуре MongoDB базе и њених колекција и докумената.

---

[13] „What is MongoDB?”, Info World, доступно: https://www.infoworld.com/article/3623357/what-is-mongodb-a-quick-guide-for-developers.html, [Последњи приступ 1. септембра 2021]

[14] „MongoDB”, Wikipedia, доступно: https://en.wikipedia.org/wiki/MongoDB, [Последњи приступ 1. септембра 2021]



Слика 7 - структура mongoDB-а

2.5 CanvasJS

CanvasJS је JavaScript и HTML5 библиотека за приказивање графичких елемената. Она је библиотека која је специјализована за област визуелизације података (енг. data visualization). Преносива је, тј. обухвата широк спектар уређаја на којима ради, а неки од њих су: iPhone, iPad, Android, Windows Phone, Microsoft Surface, рачуари са Windows OS или iOS, итд. [15]

[15] „What is CanvasJS?", CanvasJs, доступно: https://canvasjs.com/docs/charts/intro/, [Последњи приступ 2. септембра 2021]

3. ПРОЈЕКТОВАЊЕ И ИМПЛЕМЕНТИРАНА ТЕХНИЧКА РЕШЕЊА

У овом поглављу биће детаљније описана техничка решења која су примењена у оквиру конкретног пројектног задатка, са циљем да се ближе прикаже пројектни задатак самом читаоцу. Тачније, говориће се о техничком аспекту самог пројекта и детаљно ће се описати модели података и њихова међусобна интеракција.

3.1 Модели података

У овом поглављу ћемо проћи кроз моделе података и детаљније објаснити атрибуте самих класа, као и њихов циљ и намену. Бавићемо се само појединачним представљањем модела, док ћемо се у наредним поглављима бавити њиховом повезаношћу и њиховом интеракцијом.

Поглавље ће бити подељено на две логичке целине. Прва целина ће се бавити изворима хране (тј. ресурсима) док ће друга бити повезана са самим јединкама (тј. популацијом).

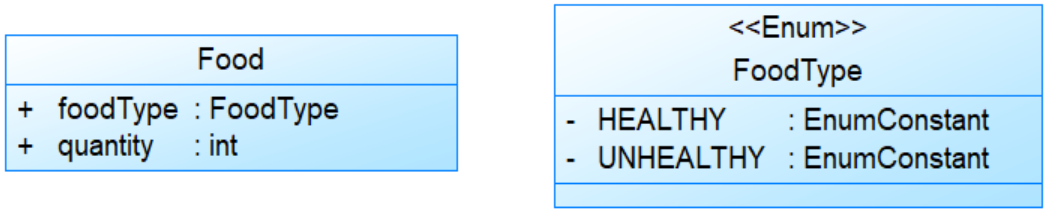
3.1.1 Храна, ресурси – извори хране

У пројектном задатку модели података који су у спрези са начином добављања ресурса су измоделовани по узору на природне изворе хране, односно, на начин на који се примитивни организми хране.

Почећемо са основном јединицом (у овом пројектном задатку класа *Food*). Јединица хране представља један конкретни елемент које јединка може да конзумира. Ако бисмо повукли паралелу са људима, до би била, на пример, једна конкретна јабука. Свака јединица хране има два кључна поља а то су: количина и квалитет.

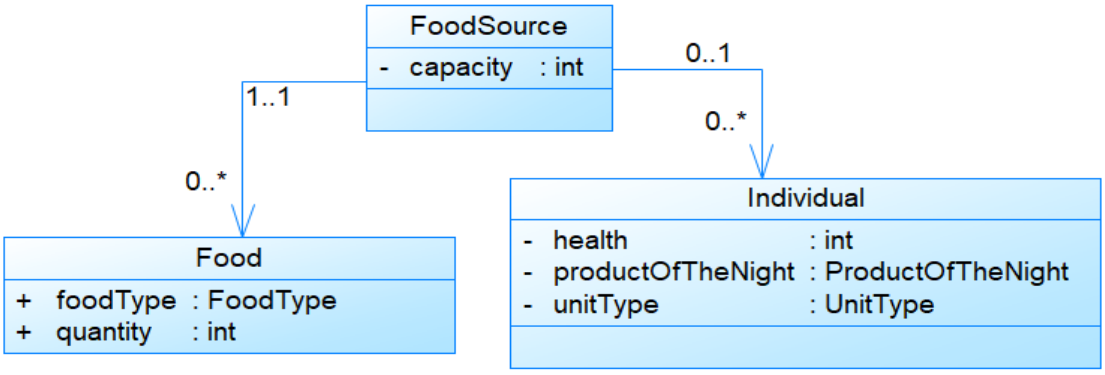
Као и у природи нема сваки извор хране исту вредност. Поље представља нутритивну вредност тј. колико конкретна храна може да задовољи потребе једне јединке у току једног циклуса храњења (нпр. једног

дана). Ако бисмо то посматрали на примеру из реалног живота, то би значило да није свака храна једнако хранљива, мада то може представљати и количину, јер је то исто битан фактор приликом исхране. Друго поље представља квалитет хране, која може бити здрава и нездрава, те се врло лако логика може проширити на комплекснију поделу квалитета. На слици 8 је приказан модел података *повезан за* јединицу хране.



Слика 8 - модел јединице хране

Други битан модел везан за ресурсе који се користе приликом исхране јесте извор хране (енг. *FoodSource*). Извор хране представља скуп више појединачних јединица храњења. На пример, то би било дрво јабуке, ако је јединица јабука или ако је јединица дрво јабуке, извор хране би био скуп више дрвећа односно воћњак. Свака јединица хране поседује две листе. Прву листу смо претходно објаснили, она представља листу јединица хране, док другу листу представљају окупатори конкретног извора хране односно јединке. Сваки извор хране може да се окупира са једном или више јединки. Такође, извор хране садржи и поље које означава капацитет самог извора, односно максималан број јединки које могу истовремено да приступе конкретном извору хране. На слици 9. је приказан модел извора хране (напомена: немојте обраћати пажњу на модел који представља јединку, он ће бити у даљем раду појашњен).

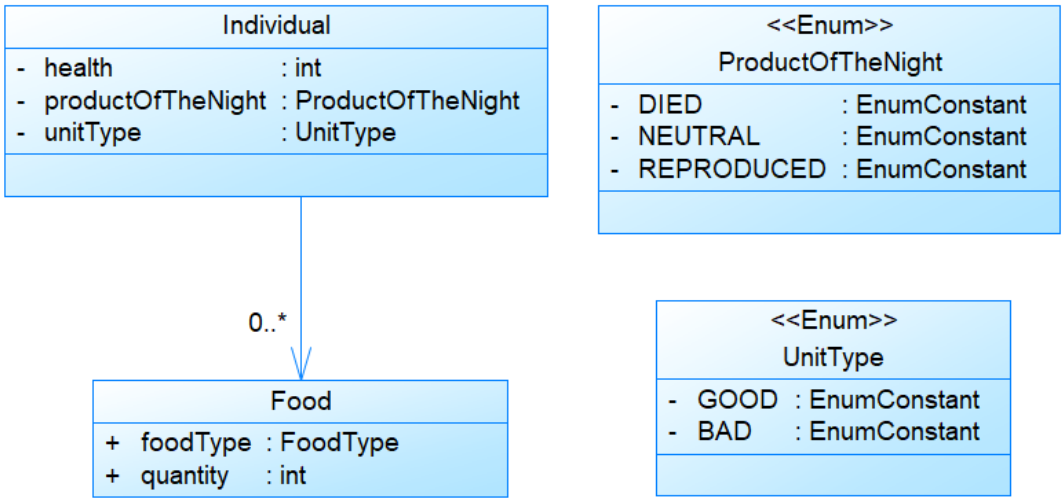


Слика 9 - модел извора хране

3.1.2 Јединка и популација

Централни део пројекта јесте део везан за јединке, односно групу једники тј. популацију. Свака јединка има следеће атрибуте: ниво енергије, тип јединке и резултат који представља вредност која настаје као продукт једног циклуса. Такође, јединка има могућност да се храни, што значи да поседује у себи и листу јединица хране коју је запосела у процесу прикупљања хране.

Ниво енергије је атрибут који директно зависи од храњења. Он представља количину хране коју је јединка унела у себе у претходној етапи тј. циклусу. Тип јединке представља карактер конкретне јединке, у пројектном задатку он је поларан, односно, јединка може бити добра или лоша. Затим, продукт ноћи, односно продук једног циклуса, је вредност која објашњава шта се десило са конкретном јединком на крају циклуса. Продукт ноћи има три могуће вредности и то су: репродуковао се, неутралан и умро. Ако јединка има вредност репродуковао се, то значи да је популација добила још једног члана, директног потомка конкретне јединке. Вредност умро, значи да је популација изгубила конкретну јединку и она ће се избацити из популације у току конкретног циклуса, док вредност неутралан представља ситуацију у којој је јединка преживела ноћ, али се није репродуковала. На крају, јединка има листу хране коју је запосела у процесу прикупљања хране. Храна коју је јединка прикупила касније помаже истој да доживи следећи циклус. На слици 10 је приказан дијаграм који графички приказује модел јединке.



Слика 10 - модел јединке

Да би постајање јединка било осигурано, односно одрживо, оне морају да имају акције које одрађују како би обнављале популацију и преживљавале. Гледајући из логичког аспекта, јединке су појединачни чланови популације које могу да се:

- Хране
- Репродукују
- Мутирају
- Умиру

Храњење јединки је кључни процес од кога зависи егзистенција јединке, такође то је процес од кога зависе сви остали процеси (осим мутације, мада и она зависи у некој мери, јер мора доћи до деобе). Јединке се хране тако што заузму конкретан извор хране, одакле узимају потребне ресурсе за преживљавање и касније га искористе. Колико ће хране узети зависи од извора хране, односно његове заузетости, тј. колико још јединки конзумира храну са конкретног извора. Такође зависи и од типа јединки с којима дели храну.

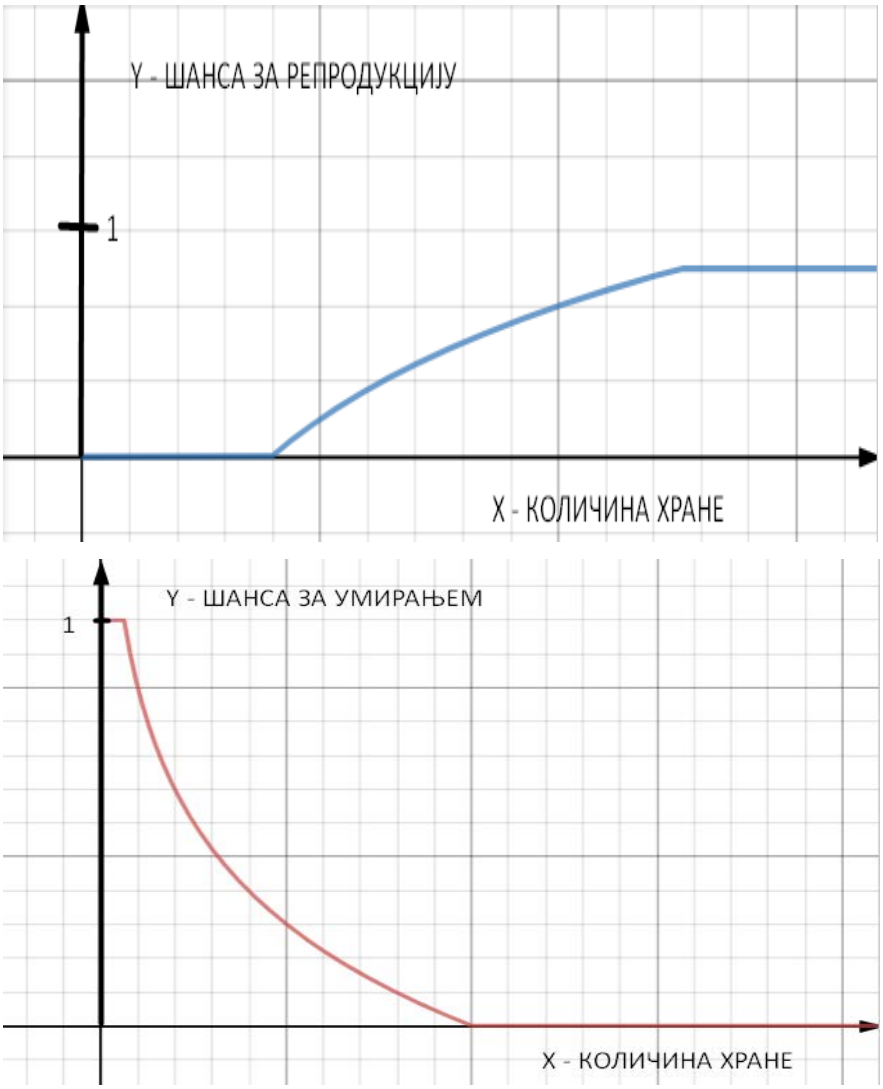
Репродукција је процес, односно акција захваљујући којој се обнавља, евентуално повећава, број популације. Репродукција јединки зависи од хране коју је јединка прикупила приликом процеса храњења. Репродукција се врши приликом завршетка циклуса. Начин на који се врши репродукција је осмишљен по моделу деоба ћелија. Од једне конкретне јединке настају две. Вероватноћа да ће доћи до деобе зависи од количине хране коју је јединка конзумирала у току ноћи. Што више хране јединка има, то је шанса за деобом, односно репродукцијом, већа.

Мутација је акција која се догађа приликом деобе. То је насумична акција која симулира мутације које се дешавају и у природи, односно промена у генетском материјалу организма. У конкретном пројектном задатку, под појмом мутације се сматра да нова јединка, настала деобом претходне јединке има шансу да буде другачији тип од претка. Јединка чији је предак припадао групи лоших јединки, може постати добра, као и јединка чији је предак припадао групи добрих јединки може се искварити. Мутације се дешавају искључиво приликом деобе и никад више.

Егзистенција јединке зависи такође од количине хране коју је јединка прикупила. Као и у природи, ако јединка нема хране, она је осуђена на умирање, тако је и у пројектном задатку, ако нема довољно хране, јединка има шансу да умре, односно ако има мање хране од минимално потребне да

би сигурно преживела ноћ. Што је ниво хране ближи нули, то је шанса за преживљавање мања. Такође, ако јединка нема довољно хране да сигурно дочека следећи циклус, онда не постоји шанса ни да се она репордукује.

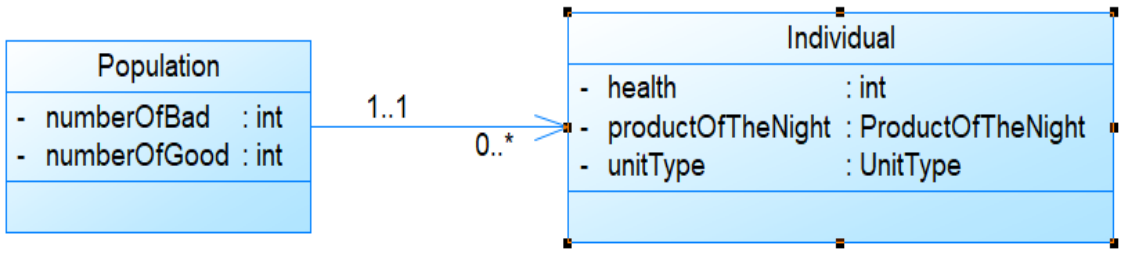
На слици 11 су приказана два графика: први који представља однос количине хране јединке на крају циклуса и њене шасе за репродукцијом и други који приказује однос количине хране и њене шансе за умирањем.



Слика 11 - графици односа количине хране и шансе за презивњавањем и шансе за умирањем

Да би јединке опстале, оне морају бити груписане. Група јединки у конкретном пројектном задатку назива се популација јединки. Популација је скуп апсолутно свих јединки које учествују у тренутној итерацији. Популација такође садржи и информације о типовима јединки унутар ње. Ове информације нису нужне, али помажу приликом имплементације и битни су за анализу. Да не бисмо пролазили стално и трагали за информацијама колико има којих јединки, овим олакшавамо и скраћујемо време извршавања програма, ако узмемо у обзир да су нам конкретан број добрих и лоших јединки битне информације за каснију анализу симулација.

Свака јединка припада једној популацији и не може бити без популације. Јединке унутар популације се међусобно боре да дођу до хране. Заправо, јединке су „глупе“ - не користе све могућности и предности које добијају тиме што припадају групи јединки. На слици 11 је приказан модел популације.

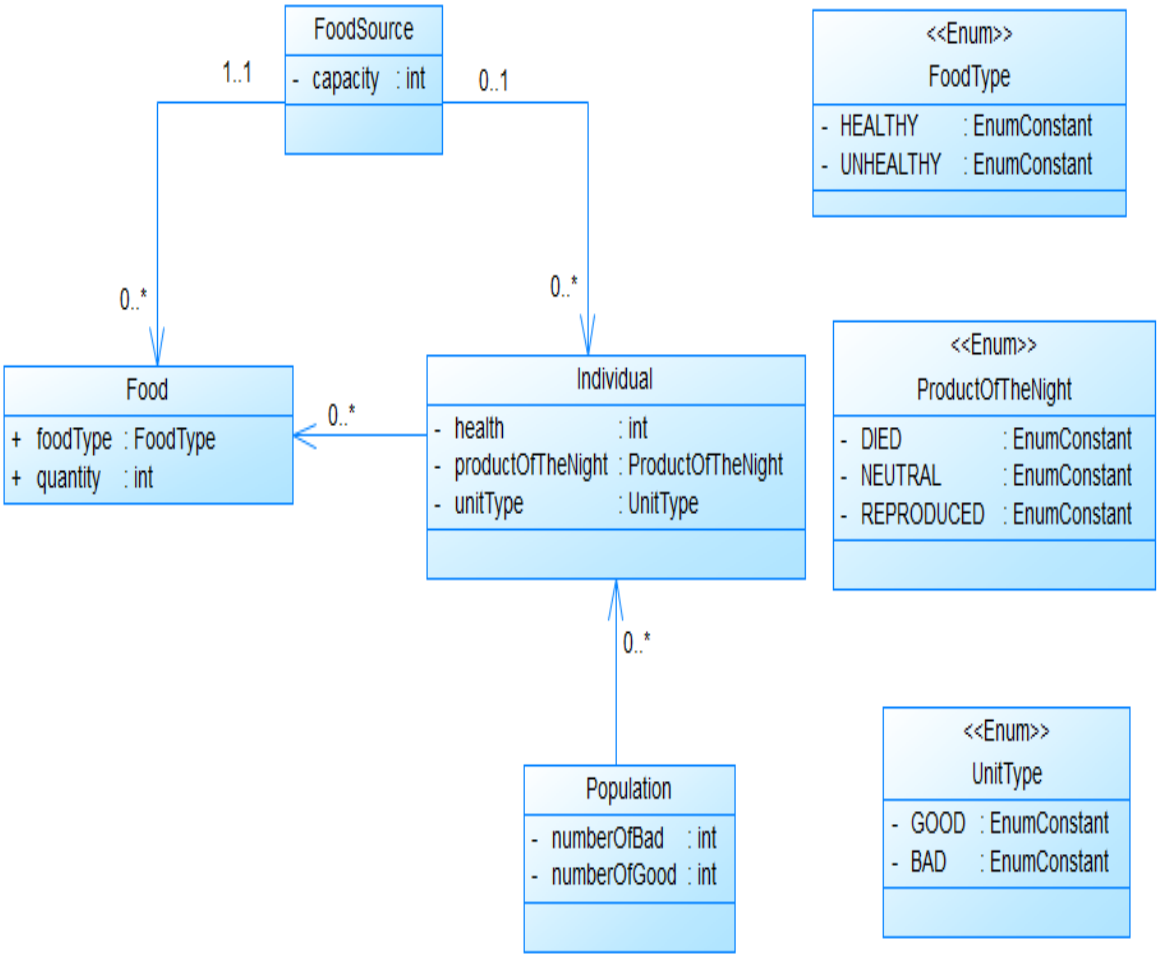


Слика 12 - модел популације

3.2 Однос и интеракција између модела

У претходном поглављу смо се бавили појединачним моделима у самом пројекту. Детаљније смо објаснили разлог њиховог постојања, њихову намену и примену. У овом поглављу ћемо обухватити све појединачне, претходно објашњене, моделе, склопити их у целину и објаснити како су они међусобно повезани, као и њихову интеракцију.

Најпре ћемо приказати дијаграм класа (eng. uml diagram), да бисмо дали прво графички приказ, ради лакшег разумевања самих односа. На слици 13 приказан је дијаграм класа.



Слика 13 – дијаграм класа

Као што видимо на дијаграму, централни део пројектног задатка је јединка. Она је у спрези са свим осталим елементима, дакле, у пројекту постоје спреге између следећих елемената:

- Популација – јединка
- Јединка – храна
- Извор хране – јединка
- Извор хране - храна

Однос популација – јединка

Однос између **популације и јединке** је осмишљен тако да јединка нема податке о другим члановима популације, као ни о самој популацији. Јединка само зна за себе и храну, док популација има информације о свим јединкама које су њени чланови. Такође, ниједна јединка не може постојати, а да буде ван популације.

Однос извор хране - јединке

Извори хране, односно хранилишта су објекти које запоседају **јединке**. Извор хранилишта води рачуна и зна ко га је запосео, па самим тим има евиденцију о јединкама које се хране на њему. На овај начин се касније на основу правила дели храна између јединки које су запоселе извор хране. Фактори који утичу на количину хране коју ће добити јединка са конкретног хранилишта су број јединки на хранилишту и типови јединки које су на хранилишту у том моменту.

Однос јединка - храна

Спрега између **јединке и хране** је кључна спрега у самом пројекту. Јединка у почетку етапе не поседује никакву храну. Јединка долази до хране у процесу сакупљања хране, а конзумира је на крају циклуса. Храна је за јединку од егзистенцијалног значаја, односно јединке које остану без хране немају начина да се прехране и осуђене су на умирање. Такође, кад јединка прикупи храну касније та иста храна не може бити дељена између јединки, она искључиво припада оној која је до ње дошла. Након храњења, храна нестаје, колико год да је има, а количина унете хране дефинише шансе за акције које се могу догодити јединки.

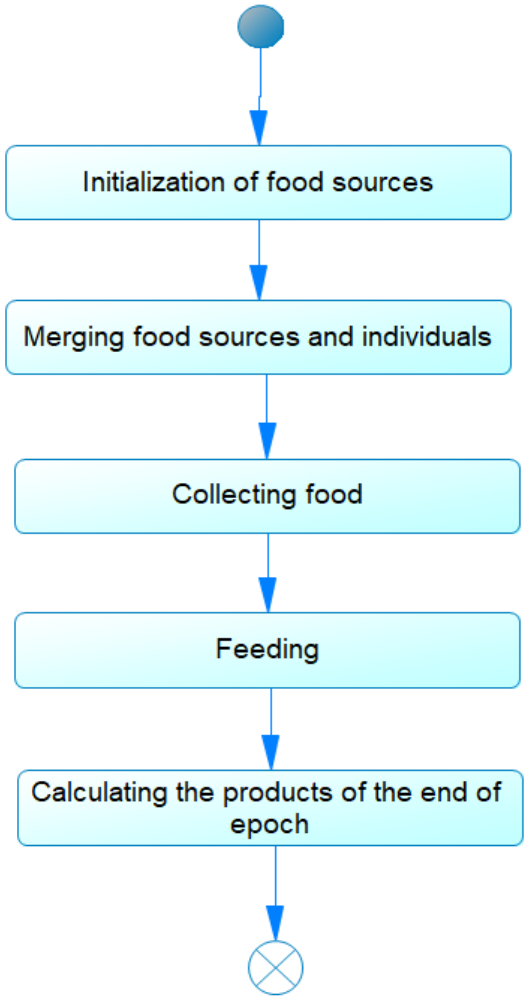
Однос извор хране – храна

Веза између **извора хране и хране** је замишљена на сличан начин као и веза између популације и јединке. Односно, извор хране поседује храну и има информације о њој, док храна нема свест о свом извору, Такође, храна иницијално не постоји ван извора хране, касније је односе јединке.

3.3 Активности

У овом поглављу ће бити објашњене појединачне активности у оквиру једне епохе, тј. циклуса. Као што је приказано на дијаграму активности на слици 14, једну епоху можемо поделити на 5 подактивности које се извршавају као у наведеном редоследу:

- стварање хране на изворишту хране
- спајања изворишта хране и јединки
- сакупљање хране
- храњење
- рачунање продукта краја епохе



Слика 14 - дијаграм активности унутар једне епохе

Процес стварања хране на изворштима хране

Приликом завршетка једне епохе јединке се хране храном која је претходно прикупљена са изворишта хране, што даље имплицира да на изворштима хране после процеса прикупљања хране, нема хране. Из наведеног се јавља потреба за обновом изворишта хране на почетку сваке итерације, у супротном јединке у итерацији не би имале хране на хранилиштима одакле је у некој од претходних итерација неко узео храну, односно животни век целокупне популације сводио би се на неколико итерација.

Процес спајања изворишта хране и јединке

За сваку јединку је најважнија ствар да дође до хране, односно да пронађе доступно изворште хране како би сакупила довољно хране да преживи до наредне итерације. Јединка у овом процесу долази до изворишта хране, односно трага за истим. Пошто је свако изворште ограничено капацитетом који означава број јединки које могу истовремено да прикупљају храну са истог, може се десити да је изворште већ претходно попуњено. У случају да су сва изворишта попуњена, јединка остаје без хране и осуђена је на умирање.

Процес сакупљања хране

Ако је јединка нашла слободно место на неком извору онда она почиње да прикупља храну. Процес прикупљања хране зависи од тога с ким дели јединка извор хране, као и од типа јединке и јединке са којом дели. Односно, да ли је јединка добра или лоша, од чега зависи којом ће се тактиком водити приликом храњења.

Постоје 4 могуће ситуације приликом храњења:

- Уколико је јединка сама на хранилишту, она ће прикупити сву доступну храну.
- Уколико су на хранилишту нађу само добре јединке, оне ће поделити на равноправне делове храну.
- Уколико се на хранилишту нађу добре и лоше јединке, онда ће лоше украсти половину хране коју су узеле добре јединке.
- Уколико се на хранилишту нађу само лоше јединке, оне ће красти међусобно и остаће са половином хране коју би узеле добре јединке, да су се нашле истој ситуацији.

Процес храњења

У процесу храњења јединке конзумирају храну коју су претходно сакупиле на хранилиштима. Процес храњења је битан, јер у њему јединке добијају енергију која им је потребна да би преживеле епоху и евентуално се репродуковале.

Процес рачунања продукта краја епохе

У процесу рачунања продуката краја епохе добија се вредност која описује шта ће се десити са јединком, односно да ли ће преживети без последица, да ли ће умрети или ће се репродуковати. Ова вредност се добија на основу енергије коју јединка поседује на крају етапе. Ниво енергије је пропорционалан са шансом да се јединка репродукује, а обрнуто пропорционалан са шансом да умре. Након што је израчуната вредност долази и до саме реализације исте, ако параметар поручује да је јединка умрла, она се брише из популације, ако се репродуковала, онда популација добија још једну јединку, а на крају ако се ништа није десило, овог последњег корака неће ни бити, тј. јединка ће само прећи у следећу епоху.

4. ПРИКАЗ И НАЧИН ИМПЛЕМЕНТАЦИЈЕ ПРОЈЕКТНОГ ЗАДАТКА

Ово поглавље ће се бавити имплементацијом конкретног пројектног задатка и претходно објашњених техничких решења. Приказ имплементације је заснован на приказу значајних делова кода, као и приказу корисничких интерфејса. Као што је раније поменуто, пројектни задатак је реализован на начин да садржи три целине и то су:

- логичка јединица одрађена у програмском језику Go,
- кориснички интерфејс за покретање и праћење симулација, одрађена у програмском језику Pharo
- кориснички интерфејс за праћење статистике

Поглавље је подељено у складу са претходно наведеним целинама. Свака целина је засебно поглавље и у њима ће се објаснити и приказати најзначајнији делови.

4.1 Go логичка јединица

У овом поглављу ћемо се бавити логичком јединицом пројекта. Логичка јединица је централни део пројекта и она је задужена за:

- покретање и извршавање симулација,
- комуникацију са корисничким интерфејсима
- уписивање логова у базу података.

4.1.1 Структура логичке јединице

Структура логичке јединице је подељена на пакете, односно логички су одвојене целине које имају сличну намену. Основни пакети у голанглоичкој јединици су следећи:

- модели

- контролери
- сервиси
- ДТО (eng. Data Transfer Object)

У пакету модели су модели који су потребни за реализацију симулације и интеракције између јединки и хране.

Листинг 1 приказује имплементацију класа модела хране и извора хранилишта у програмском језику Go. Нотација JSON дефинише назив атрибута приликом конверзије из Go објекта у JSON објекат.

```
package models

type FoodType int

const (
    HEALTHY   FoodType = 0
    UNHEALTHY FoodType = 1
)

type Food struct {
    FoodType FoodType `json:"food-type"`
    Quantity int      `json:"quantity"`
}

func NewFood(foodType FoodType, quantity int) *Food {
    food := new(Food)
    food.FoodType = foodType
    food.Quantity = quantity
    return food
}

type FoodSource struct {
    Resources  []*Food
    OccupiedBy []*Individual
    Occupied   int
}
```

Листинг 1 - класе модели хране и извора хране

Контролери су имплементирани по REST API шаблону. Рутер је постављен на порту 8080 и користи http протокол за комуникацију.

Крајње тачке (енг. endpoints) су подељени у две групе. Први су задужени за комуникацију са Pharo корисничким интефејсом, док су други задужени за комункицију са корисничким интерфејсом који се бави статистиком. Приликом имплементације апија коришћен је *gin* радни оквир. На листингу 2 је приказана имплементација рутера и његовог CROS Middleware.

```
var r = gin.Default()
func CORSMiddleware() gin.HandlerFunc {
    return func(c *gin.Context) {
        c.Writer.Header().Set( key: "Access-Control-Allow-Origin", value: "*")
        c.Writer.Header().Set( key: "Access-Control-Allow-Credentials", value: "true")
        c.Writer.Header().Set( key: "Access-Control-Allow-Headers",
            value: "Content-Type, Content-Length, Accept-Encoding, X-CSRF-Token," +
            " Authorization, accept, origin, Cache-Control, X-Requested-With")
        c.Writer.Header().Set( key: "Access-Control-Allow-Methods", value: "POST, " +
            "OPTIONS, GET, PUT")
        if c.Request.Method == "OPTIONS" {
            c.AbortWithStatus( code: 204)
            return
        }
        c.Next()
    }
}
func initRouter() {
    r.Use(CORSMiddleware())
    r.GET( relativePath: "/api/toTheEnd", controllers.ToTheEnd)
    r.POST( relativePath: "/api/setGlobals", controllers.SetGlobals)
    r.POST( relativePath: "/api/initData", controllers.SetInitData)
    r.GET( relativePath: "/api/nextEpoch", controllers.NextEpoch)
    r.GET( relativePath: "/api/currentData", controllers.CurrentData)
    r.GET( relativePath: "/api/mainData", controllers.GetMainData)
    r.GET( relativePath: "/api/fullData", controllers.GetFullData)
    r.GET( relativePath: "/api/firstData", controllers.GetFirstData)
    r.GET( relativePath: "/api/fullData/:id", controllers.GetFullDataById)
    r.Run()
}
```

Листинг 2 - имплементације рутера и његовог CROS middlewara-а

ДТО објекти су објекти који се користе приликом слања и примања података. Налик су објектима у моделу, само уз неке прктичне измене које олакшавају слање и примање података.

Сервиси служе за директан рад са симулацијом и за манипулацију података приликом комуникације са репозиторијумима из MongoDB базе података. Сервиси ће бити детаљније објашњени у наредним поглављима, кад буде речи о акцијама и паралелизацији, као и о самом раду са базом и логовима.

4.1.2 Занчајне функције

Симулација, односно акције које се догађају приликом симулације и у итерацијма су круцијални део пројекта, у овом делу рада ћемо се бавити њима.

Акције се налазе у сервисима, односно сервиси их покрећу. Такође акције су паралелизоване уз помоћ *goroutines.* У наставку ћемо приказати неке од најзначајнијих функција.

Почећемо од мање компликоване функције. На листингу 3 је приказан процес обнове и извора хране. Она се извршава у паралели у групама величине 16, то је омогућује waitGroup објекта, који регулише конкурентност.

```
func ClearFoodSourcesParallel(foodSources []*models.FoodSource, wg *sync.WaitGroup) {
    var timesLoop = int(len(foodSources) / globals.ParallelDayInterval)
    for i := 0; i < timesLoop; i++ {
        wg.Add(globals.ParallelDayInterval)
        for j := 0; j < globals.ParallelDayInterval; j++ {
            go clearSourceParallel(i*globals.ParallelDayInterval+j,
                foodSources[i*globals.ParallelDayInterval+j], wg)
        }
        wg.Wait()
    }
    var restFood = len(foodSources) % globals.ParallelDayInterval
    wg.Add(restFood)
    for i := timesLoop * globals.ParallelDayInterval; i < len(foodSources); i++ {
        go clearSourceParallel(i, foodSources[i], wg)
    }
    wg.Wait()

}
```

Листинг 3 - функција обнове извора

На листингу 4 је приказана функција која у паралели решава конфликте настале на хранилиштима. Борба око хране и подела хране је реализована на следећи начин:

- Ако је јединка сама нахранилишту, она ће прикупити сву доступну храну.
- Ако се на хранилишту нађу две добре јединке, оне ће поделити на равноправне делове храну.
- Ако се на хранилишту нађу добра и лоша јединка, онда ће лоша украсти половину хране коју је добра јединка узела.
- Ако се на хранилишту нађу две лоше јединке, оне ће красти међусобно и остаће са половином хране коју би узеле добре јединке да су се нашле истој ситуацији.

```
func giveFoodToPlebParallel(foodSource *models.FoodSource, wg *sync.WaitGroup) {
    if foodSource.Occupied == 0 {
        defer wg.Done()
        return
    }
    if foodSource.Occupied == 1 {
        foodSource.OccupiedBy[0].Health = len(foodSource.Resources) * 100
        defer wg.Done()
        return
    }
    if foodSource.OccupiedBy[0].TypeOfIndividual == models.GOOD &&
        foodSource.OccupiedBy[1].TypeOfIndividual == models.GOOD {
        var food = len(foodSource.Resources) / 2.0 * 100.0
        foodSource.OccupiedBy[0].Health = food
        foodSource.OccupiedBy[1].Health = food
    } else if foodSource.OccupiedBy[0].TypeOfIndividual == models.BAD &&
        foodSource.OccupiedBy[1].TypeOfIndividual == models.GOOD {
        var food = len(foodSource.Resources) / 2.0 * 100.0
        foodSource.OccupiedBy[0].Health = food + food/2
        foodSource.OccupiedBy[1].Health = food / 2
    } else if foodSource.OccupiedBy[0].TypeOfIndividual == models.GOOD &&
        foodSource.OccupiedBy[1].TypeOfIndividual == models.BAD {
        var food = len(foodSource.Resources) / 2.0 * 100.0
        foodSource.OccupiedBy[0].Health = food / 2
        foodSource.OccupiedBy[1].Health = food + food/2
    } else {
        var food = len(foodSource.Resources) / 2.0 * 100.0
        foodSource.OccupiedBy[0].Health = food / 2
        foodSource.OccupiedBy[0].Health = food / 2
    }
    defer wg.Done()
}
```

Листинг 4 - функција решавања конфликта на хранилиштима

На листингу 5 је приказана функција која рачуна параметар краја итерације, који дефинише шта се десило са јединком. Догађаји се добијају на основу шансе, а резултат шансе се добија помоћу функције *calculateIsHappened.*

CalculateIsHappened се позива у два случаја, када нема максимално енергије јединка (у овом случају 100), тада се проверава да ли че умрети. А други случај када се позива је када има максимално енергије, тада се насумично на осноу енергије проверава да ли ће преживети.

```
func nightTimeLogicParallel(individual *models.Individual, wg *sync.WaitGroup) models.ProductOfTheNight {
    if individual.Health <= 0 {
        individual.NightRes = models.DIED
        defer wg.Done()
        return models.DIED
    }
    if individual.Health < MaxHP {
        if calculateIsHappened(MaxHP - individual.Health + globals.BoostToSurvive) {
            individual.NightRes = models.DIED
            defer wg.Done()
            return models.DIED
        }
    } else if individual.Health >= MaxHP {
        if calculateIsHappened(individual.Health - MaxHP + globals.BoostToSurvive) {
            individual.NightRes = models.REPRODUCED
            defer wg.Done()
            return models.REPRODUCED
        }
    }
    individual.NightRes = models.NEUTRAL
    defer wg.Done()
    return models.NEUTRAL
}

func calculateIsHappened(marginalValue int) bool {
    if rand.Intn(MaxHP) > marginalValue { false }
    return true
}
```

Листинг 5 - функција рачунања параметара краја итерације

Последња функција коју ћемо приказати је функција расподеле јединки на изворištима. Она функционише на следећи начин. Имамо у старту две листе, листу јединки које нису распоређене и листу изворишта која нису попуњена. У старту су обе листе напуњене свим јединкама, односно свим изворištима. Затим се итерира кроз петљу, док не буду сва изворишта попуњена или докле год не буду распоређене све јединке (односно док нека од листи не буде празна). Онда се бира насумичан елемент из обе листе и распоређују се, тј. спајају. Затим се јединка уклања из листе слободних јединки, док се проверава да ли је попуњен извор хране, ако јесте и он се уклања из листе слободних изворишта. Ово се понавља у итерацијама док се не задовољи један од два услова за излазак из петље. На листингу 6 је приказана имплементација претходно описане функције.

```
func Merge(population []*models.Individual, foodSources []*models.FoodSource) {
    var populationFreeArray = utils.MakeArray(len(population))   //Lista id-ova
    var foodSourcesFreeArray = utils.MakeArray(len(foodSources)) //Lista id-ova
    // prolazim kroz listu dok svi ne nadju izvori za hranu ili dok se ne popune svi izvori hrane
    for len(populationFreeArray) != 0 {
        // posto lista sadrzi sve id-jove uzmemo random iz nje
        if len(foodSourcesFreeArray) == 0 {
            break
        }
        var foodRandomId = rand.Intn(len(foodSourcesFreeArray))
        var foodSourceId = foodSourcesFreeArray[foodRandomId] //random id samo
        var individualRandomId = rand.Intn(len(populationFreeArray))
        var plebId = populationFreeArray[individualRandomId]
        var pleb = population[plebId]
        if foodSources[foodSourceId].Occupied == 0 {
            foodSources[foodSourceId].OccupiedBy = append(foodSources[foodSourceId].OccupiedBy, pleb)
            foodSources[foodSourceId].Occupied++
        } else {
            foodSources[foodSourceId].OccupiedBy = append(foodSources[foodSourceId].OccupiedBy, pleb)
            foodSources[foodSourceId].Occupied++
            foodSourcesFreeArray = append(foodSourcesFreeArray[:foodRandomId],
                foodSourcesFreeArray[foodRandomId+1:]...)
        }
        populationFreeArray = append(populationFreeArray[:individualRandomId],
            populationFreeArray[individualRandomId+1:]...)
    }
}
```

Листинг 6 - функција спајања изворишта и јединка

4.1.3 Паралелизам и његов ефекат

Функције у пројекту су паралелизоване уз помоћ goroutines и waitGropus. Приликом паралелизације сусрели смо се са проблемом који ствара сама природа итерација симулације, односно акције унутар итерације су се морале извршавати серијиски. То је решено коришћењем waitGropus објеката, свака итерација има своју waitGropus која води рачуна да се ствари изврше редоследно, заправо она служи у овом конкретном случају да чека да се заврше све акције у паралели које су покренуте пре ње.

WaitGropus је била значајан фактор код још једног проблема. Наиме, неке акције не захтевају превелики рад и самим тим њихова паралелизација може само успорити време извршавања програма, због проблема синхронизације. Са друге стране, природа goroutines објеката је таква да даје могућност да се позове у паралели неодређен број функција, са којима он после интерно ради. Међутим, уочено је да велики број позива функција додатно успорава рад програма.

После истраживања и експериментисања, уочено је да је најоптималније решење и за то примена, већ споменутих, waitGropus објеката, који би ограничили број група који се позивају приликом итерација. На пример уместо да се позове одмах 500 goroutines, позваће се 16 goroutines паралелно, а затим тако још 31 пут и на крају још 4 која су остала да се изврше.

Након дужег израживања уочено је да величина групе не даје исте пеформансе на ралзичитим симулација, па је одлучено да се групе подесе у зависности од броја иницијалних изворшта хране на следећи начин:

- Ако је број извора хране мањи од 2000 група је величине 300.
- Ако је број извора хране мањи од 6000 величина групе је једна петнаестина изворишта хране.
- У осталим случајевима група има величину 16.

На слици 15 су приказани добијени подаци приликом покретања симулације. Подаци представљају просечно време прилком 100 покренутих симулација, где је број итерација био 50. Прва колона преставља број изворшта хране, а друге две време у секундама потребно за извршавање једне симулације. Уочава се да су параметри подешени тако да највеће убрзање програм има кад се окреће у опсегу од 10.000 до 25.000 изворшта хране, а затим убрзање постаје константно за веће случајеве од 25.000 изворшта.

Number of food sources	Parallel (sec)	NotParallel (sec)
100000	580.2	812.9
75000	360.1	400.8
50000	39.2	52.5
25000	9.7	18.1
20000	7.2	16.7
15000	4.7	13.0
10000	3.2	6.1
5000	1.0	1.1

Слика 15 - табела ефекта паралелизације

Паралелизација је тестирана на лаптоп рачунару следећих спецификација:

- Порцесор: Intel Core i7 @ 1.80GHz
- Рам меморија: 16.0GB
- Графичка картица: AMD Radeon 610 Series (Dell)
- Ром меморија: 256GB SSD

4.1.4 Интеракција са базом и логовима

Да бисмо имали увид у статистику саме симулације и након што је она завршена, морамо податке о конкретној итерацији негде и да складиштимо. У те сврхе нам служи NoSql база MongoDB.

MongoDB је, као што је већ речено, документ орјентисан NoSQL база података, односно база података чија је намена да чува документе. Документи унутра коришћене базе су складиштени у JSON формату. У пројектном задатку имамо две врсте докумената који се складиште у две различите колекције, а то су:

- Главни параметар о симулацијама
- Параметри о комплетној симулацији

Главни параметри о симулацији представљају само основне параметре о самој симулацији као што су: број извора хране, ниво симулације и датум покретања, као и финална епоха (колико итерација има цела симулација). Разлог посебног складиштења ових података је тај што су

нам ови подаци често коришћени (кад бирамо симулацију коју ћемо отворити да бисмо видели њену статистку, приказују се ови подаци у корисничком интерфејсу), па да не би програм стално приступао масивним, гломзним подацим, решено је да се главни подаци одвоје у посебну колекцију.

Парамтри о комплетној симулацији су нам потребни да бисмо анализирали конкретну симулацију. Они се чувају у JSON документу са следћим пољима: главни подаци (дупликат претходно описаних података) и подаци о свакој епохи засебно, Подаци о епохама су следећи: број конкретне епохе, број добрих јединки у конкретној епохи и број лоших јединки у конкретној епохи.

На листингу 7 је приказана структура JSON фајла који саджи комплетне информације.

```
{
    "_id": "611fd12d37b42a9a145ece7b",
    "MainParams": {
        "_id": "611fd12d37b42a9a145ece7b",
        "startDate": "2021/08/19 01:49:52",
        "NumberOfFoodSources": 100,
        "FinalEpoch": 100,
        "Level": "easy"
    },
    "Epoches": [
        {
            "CurrentEpoch": 0,
            "NumberOfGood": 5,
            "NumberOfBad": 1
        },
        {
            "CurrentEpoch": 1,
            "NumberOfGood": 8,
            "NumberOfBad": 2
        },
        ...]
}
```

Листинг 7 - приказ структуре JSON фајла са комплетним информацијама

Кад знамо шта имамо од датотека у бази, треба да сазнамо како се оне стварају, па у те сврхе нам служи лог фајл, односно дневник догађаја. Док се симулација извршава, подаци се уписују у лог фајл. Након завршетка симулације, лог фајл се конвертује у JSON формат који се уписује у бази података. Линија у лог фајлу садржи информације о: времену кад је записан лог, типу информације (линије), тренутну епоху, крајњу епоху, број извора хране, број добрих јединки, број лоших јединки и ниво симулације.

Пример линије у лог фајлу:

2021/09/08 22:37:52 [INFO] CurrentEpoch: 31, FinalEpochs: 50, NumberOfFoodSources: 250, NumberOfGoods: 102, NumberOfBads: 333, Level: easy

Комуникација са лог фајлом и базом се врши преко њихових сервиса, који су задужени за манипулацију са подацима и конекцијом. На листингу 8 је приказана имплементација функције која уписује податке у MongoDB базу.

```
func WriteInMongoDB(document *dto.MongoDocumentDto) {
    connectDB()
    ctx, cancelFun := context.WithTimeout(context.Background(), 10*time.Second)
    fullDataCollection := globals.MongoDBConnection.Collection(_const.FullDataCollection)
    mainParamsCollection := globals.MongoDBConnection.Collection(_const.MainCollectionName)

    _, fullDataCollectionErr := fullDataCollection.InsertOne(ctx, document)
    go func() {
        if fullDataCollectionErr != nil {
            cancelFun()
            log.Fatal( v...: "fullDataCollectionErr")
        }
    }()

    _, mainParamsCollectionErr := mainParamsCollection.InsertOne(ctx, document.MainParams)

    if mainParamsCollectionErr != nil {
        log.Fatal( v...: "mainParamsCollectionErr")
    }
}
```

Листинг 8 – функција која уписује фајла у mongoDB

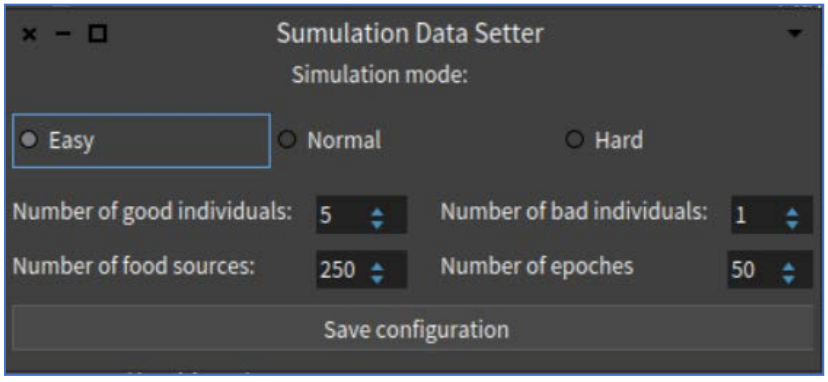
4.2 Pharo кориснички интерфејс

У овом поглављу биће приказан Pharo кориснички интерфејс и објашњен начин на који је имплементиран. Pharo кориснички интерфејс је имплементиран из две целине: део за унос параметара симулације и део за праћење тока саме симулације.

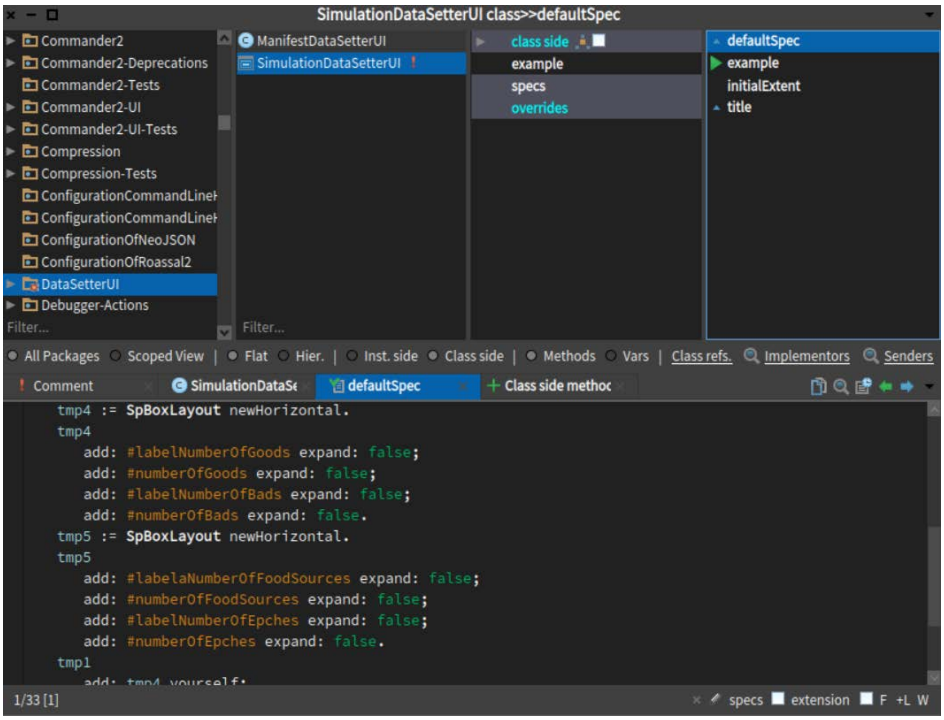
4.2.1 Кориснички интерфејс за унос параметара

Кориснички интерфејс за унос параметара је имплементиран уз помоћ Spec, оквира за развоје (енг. framework) који је израђен програмском језику Pharo за изградњу корисничких интерфејса. Креирана је класа *SimulationDataSetterUI* (слика 17) која гради кориснички интерфејс (слика 16). Она у себи садржи:

- Поље за унос броја иницијалних добрих јединки
- Поље за унос броја иницијалних лоших јединки
- Поље за унос броја извора бране
- Поље за унос броја епоха које ће се симулирати
- Опциона дугмад за бирање нивоа симулације



Слика 16 - кориснички интерфејс за унос параметара симулације



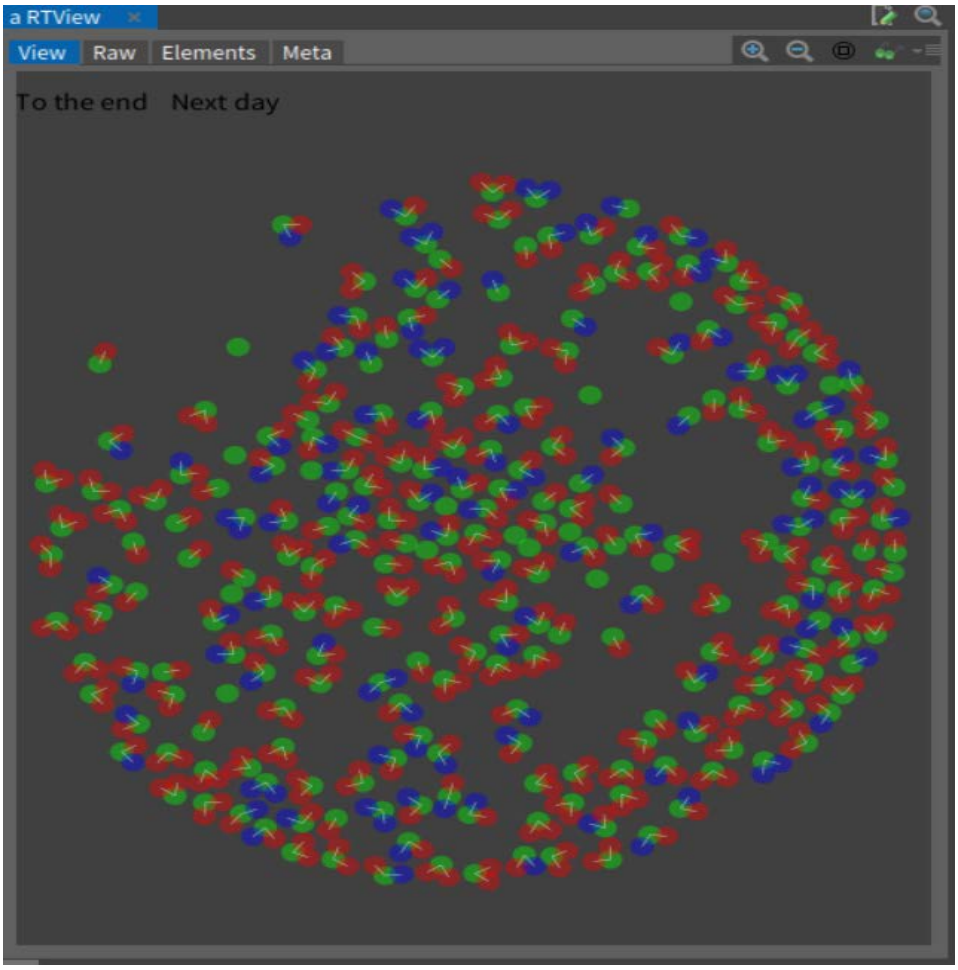
Слика 17 - део имплементације SimulationDataSetterUI класе у pharo окружењу

4.2.2 Кориснички интерфејс за праћење симулације

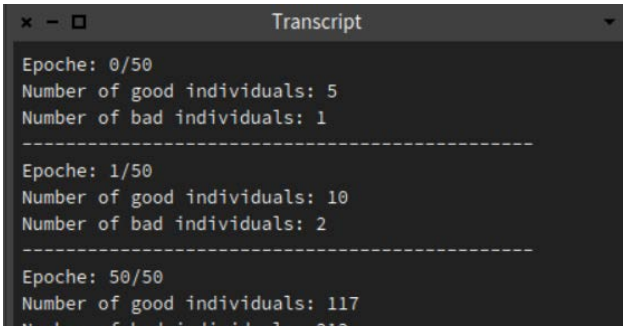
Кориснички интерфејс за праћење симулације је реализован уз помоћ roassal библиотеке за визуелизацију података. Подаци су визуализовани на следећи начина:

- Плави круг представља добру јединку
- Црвени круг представља лошу јединку
- Зелени круг представља извор хране

Кругови који представљају јединке су повезани са изворištем хране које је окупирано. Постоје и два дугмета која контролишу ток симулације, једно је за одлазак у следећу фазу, а друго је за аутоматско симулирање до последње фазе. На слици 18. је приказан кориснички интерфејс за контролу тока трансакције, такође постоји и транскрипт у коме се уписују подаци о итерацијама које су симулиране (слика 19).



Слика 18 - кориснички интерфејс за приказ и контролу тока симулације



Слика 19 - приказ транскрипта који приказује информације о симулираним епохама

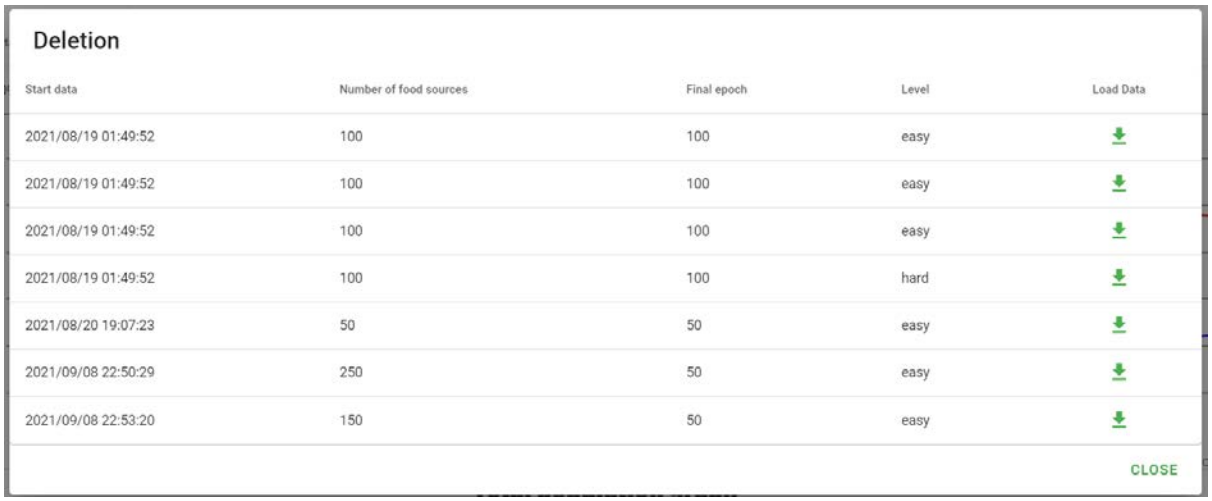
4.3 Веб приказ статистике

Веб приказ статистике представља веб апликацију која се покреће на localhost и порту 4201. Одрађен је у Vue.js - MVVM (Model–view–viewmodel) frontend javascript оквиру (енг. framework) отвореног кода (енг. open source).

Веб приказ статистике нуди следеће ствари:

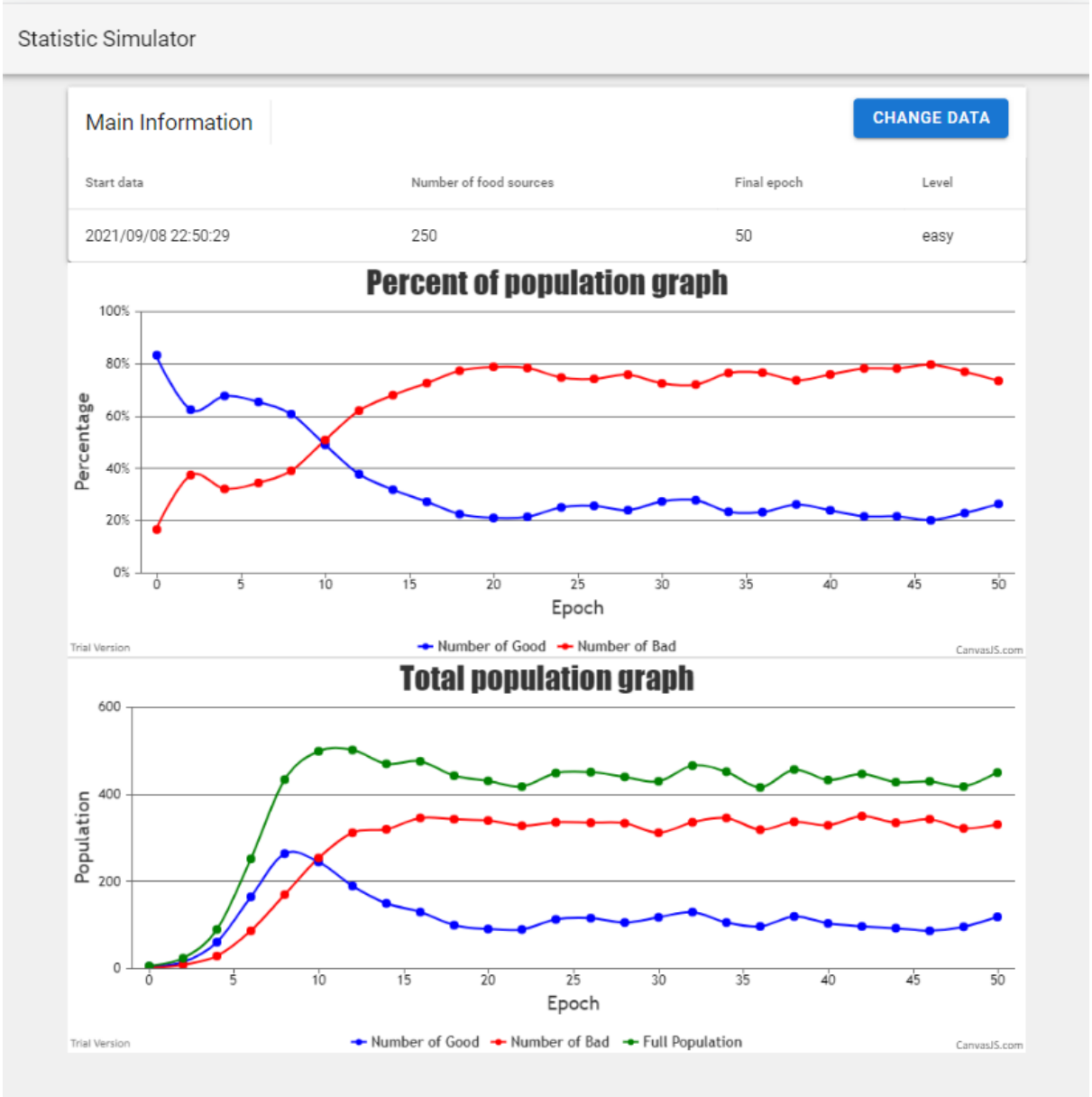
- Приказ главних података о симулацији
- Приказ графика са подацима о броју добрих, лоших и укупног броја јединка кроз етапе
- Приказ графика са процентуалним односом добрих и лоших јединки кроз етапе
- Могућност отварања било које симулације која се налази и сачувана је у бази (слика 20)

На слици 20 је приказана табела која нуди могућност учитавања података од било којој симулацији, притиском на зелено дугме у крајње десној колони.



Слика 20 - табела главних података која нуди могућност учитавња података од симулацији

На слици 21. приказан је основни изглед веб апликације за приказ статистике. На графицима плаве линије представљају добре јединке, црвене представљају лоше јединке, док је зелена линија збир добрих и лоших јединки. Притиском на дугме “CHANGE DATA”, отвара се табела са слике 20.



Слика 21 - основни приказ веб апликације

5. ЗАКЉУЧАК

Рад је настојао да кроз три кључна аспекта објасни принцип функционисања симулације преживљавања јединки. Преживљавање је дефинисано као процес испуњавања елементарних тј. егзистенцијалних потреба група јединки, где се конкретно мислило на исхрану и репродукцију. Фокус је био усмерен ка што адекватнијој употреби постојећих ресурса, при чему то не би довело до негативних последица по здравље јединки, односно не би ометало испуњење основних животних потреба, што би потенцијално могло довести до смрти јединке.

Дакле, наведено је да су у раду постојале три етапе, од којих су се две позабавиле приказом корисничког интерфејса, док је трећа имала улогу у објашњавању поступка прикупљања, обраде и складиштења података. Детаљно се обрађују две технологије, у које су биле имплементиране две врсте корисничких интерфејса: Pharoa и Vue.js. Уз Голанг дошло је до реализације самог извршења симулације, слоја за манипулацију, али и Дневника догађаја. Кроз поменути Дневник догађаја биле су забележене и меморисане информације о догађајима који су се одиграли за време рада програма, те су тако записане колекције информација трајно сачуване на меморијском медијуму.

Друго поглавље било је од немерљиве важности, у погледу разумевања коришћених технологија у самом пројекту. Све технологије које су биле у употреби приликом израде овог пројекта, прецизно су одређене неопходним и посебно важним информацијама и описима.

Конкретизација модела података и њихове међусобне интеракције одвија се у трећем поглављу рада, те су се ту нашли и прецизни описи техничких решења и начина њихове примене. Намера је била да се помоћу дијаграма ближе објасне подузете активности у процесу преживљавања.

На крају, јасно је објашњена имплементација пројектног задатка, са пажњом усмереном на визуелни приказ имплементације. Поменути приказ односио се на истакнуте делове кода и корисничког интерфејса.

6. ЛИТЕРАТУРА

[1] Robert Griesemer, „The Evolution of Go”, Google Inc, 2015.

[2] „Frequently Asked Questions (FAQ)”, Golang, доступно: https://golang.org/doc/faq, [Последњи приступ 31. августа 2021]

[3] „What is Gin in Golang?”, Eduactive, доступно: https://www.educative.io/edpresso/what-is-gin-in-golang, [Последњи приступ 31. августа 2021]

[4] „Golang Concurrency”, Golang Programs, доступно: https://www.golangprograms.com/go-language/concurrency.html, [Последњи приступ 31. августа 2021]

[5] Oscar Nierstrasz, Stéphane Ducasse, Damien Pollet, „Pharo by Example”, Lulu. com, 2010.

[6] Alexandre Bergel, Damien Cassou, Stéphane Ducasse, Jannik Laval, „Deep into Pharo”, Lulu. com, 2013.

[7] Johan Fabry, Stéphane Ducasse, „The Spec UI framework”, Square Bracket Associates, 2017.

[8] Sven van Caekenberghe, „Zinc: an industrial quality level HTTP/S framework”, Square Bracket Associates, 2020.

[9] Macrae, Callum, „Vue. js: Up and Running: Building Accessible and Performant Web Apps”, O'Reilly Media, Inc, 2018.

[10] Nelson, Brett, „Getting to Know Vue. js”, Apress, 2018.

[11] Andrea Koutifaris, „Vuex Quick Start Guide: Centralized State Management”, O'Reilly Media, Inc, 2018.

[12] „Introduction - What is Vuetify?”, VuetifyJs, доступно: https://vuetifyjs.com/en/introduction/why-vuetify/, [Последњи приступ 1. септембра 2021]

[13] „What is MongoDB?”, Info World, доступно: https://www.infoworld.com/article/3623357/what-is-mongodb-a-quick-guide-for-developers.html, [Последњи приступ 1. септембра 2021]

[14] „MongoDB”, Wikipedia, доступно: https://en.wikipedia.org/wiki/MongoDB, [Последњи приступ 1. септембра 2021]

[15] „What is CanvasJS?”, CanvasJs, доступно: https://canvasjs.com/docs/charts/intro/, [Последњи приступ 2. септембра 2021]

УНИВЕРЗИТЕТ У НОВОМ САДУ ● **ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА**
21000 НОВИ САД, Трг Доситеја Обрадовића 6

КЉУЧНА ДОКУМЕНТАЦИЈСКА ИНФОРМАЦИЈА

Редни број, **РБР**:	
Идентификациони број, **ИБР**:	
Тип документације, **ТД**:	Монографска документација
Тип записа, **ТЗ**:	Текстуални штампани материјал
Врста рада, **ВР**:	Завршни (bachelor) рад
Аутор, **АУ**:	Дамјан Манојловић
Ментор, **МН**:	др Игор Дејановић
Наслов рада, **НР**:	Симулација преживљавања популације јединки
Језик публикације, **ЈП**:	српски/латиница
Језик извода, **ЈИ**:	српски/енглески
Земља публиковања, **ЗП**:	Република Србија
Уже географско подручје, **УГП**:	Војводина
Година, **ГО**:	2021.
Издавач, **ИЗ**:	Ауторски репринт
Место и адреса, **МА**:	Нови сад, трг Доситеја Обрадовића 6
Физички опис рада, **ФО**: (поглавља/страна/ цитата/табела/слика/графика/прилога)	6/43/15/0/21/0/0
Научна област, **НО**:	Електротехничко и рачунарско инжењерство
Научна дисциплина, **НД**:	Примењене рачунарске науке и информатика
Предметна одредница/Кључне речи, **ПО**:	Наредне технике програмирања
УДК	
Чува се, **ЧУ**:	У библиотеци Факултета техничких наука, Нови Сад
Важна напомена, **ВН**:	
Извод, **ИЗ**:	У раду је детаљно описана имплементација и начин функционисања симулације преживљавања јединки, са нагласком на паралелизацији етапа и накнадном праћењу и анализирању симулација. Платформа је реализована уз ослонац на програмске језике Go и Pharo и коришћење vue.js библиотеке за подршку праћења статистике на вебу.
Датум прихватања теме, **ДП**:	
Датум одбране, **ДО**:	
Чланови комисије, **КО**: Председник:	др Милан Видаковић, редовни професор
Члан:	др Жељко Вуковић, доцент
Члан, ментор:	др Игор Дејановић, ванредни професор

Потпис ментора

UNIVERSITY OF NOVI SAD ● **FACULTY OF TECHNICAL SCIENCES**
21000 NOVI SAD, Trg Dositeja Obradovića 6

KEY WORDS DOCUMENTATION

Accession number, **ANO**:	
Identification number, **INO**:	
Document type, **DT**:	Monographic publication
Type of record, **TR**:	Textual printed material
Contents code, **CC**:	Bachelor Thesis
Author, **AU**:	Damjan Manojlović
Mentor, **MN**:	Igor Dejanović, Ph.D, assoc. prof.
Title, **TI**:	A simulation of population survival
Language of text, **LT**:	Serbian
Language of abstract, **LA**:	Serbian
Country of publication, **CP**:	Republic of Serbia
Locality of publication, **LP**:	Vojvodina
Publication year, **PY**:	2021.
Publisher, **PB**:	Author's reprint
Publication place, **PP**:	Novi Sad, Dositeja Obradovica sq. 6
Physical description, **PD**: (chapters/pages/ref./tables/pictures/graphs/appendixes)	6/43/15/0/21/0/0
Scientific field, **SF**:	Electrical and Computer Engineering
Scientific discipline, **SD**:	Applied computer science and informatics
Subject/Key words, **S/KW**:	Advanced Programming Techniques
UC	
Holding data, **HD**:	The Library of Faculty of Technical Sciences, Novi Sad, Serbia
Note, **N**:	
Abstract, **AB**:	The thesis describes the implementation and functionality of survival simulation of an individual, with a focus on parallelization of survival stages and the subsequent analysis of the received results. The platform was implemented using Go and Pharo programming languages, along with the Vue.js library, which was used for a web-based preview of survival statistics.
Accepted by the Scientific Board on, **ASB**:	
Defended on, **DE**:	

Defended Board, **DB**:	President:	Milan Vidaković, Ph.D, full prof.
	Member:	Željko Vuković, Ph.D, assist. prof.
	Member, Mentor:	Igor Dejanović, Ph.D, assoc. prof.

Menthor's sign